ForgeAI

System design: high-level design, low-level design and flows

A self-hosted platform for fine-tuning, evaluating, deploying, serving and operating small language models, operated by an AI SRE agent that diagnoses incidents and remediates them after human approval.

Field	Value
Document	System design: HLD, LLD, flows
Version	0.1
Date	5 October 2026
Status	Implemented. Local stack verified end to end; GPU / Kubernetes paths not yet exercised
Codebase	forge/ (Python 3.11): ~4,100 lines across 24 modules; 35 tests
Audience	Engineers reviewing or extending the platform

Contents

1. Overview
2. High-level design (HLD)
3. Low-level design (LLD)
4. Flows
5. Verification status and known limits

1. Overview

ForgeAI turns the model lifecycle into one system: **train → evaluate → register → deploy → serve → observe → diagnose → approve → remediate → verify**. The agent is not the product. It is the platform's operator, and it acts only through the same API, RBAC, audit log and idempotency guarantees as a human operator.

1.1 Goals

- Produce a better model from a dataset (LoRA / QLoRA SFT) with full data lineage.
- Answer “is v2 actually better than v1?” across quality, latency and cost, and turn the answer into an explicit ship decision.
- Serve models through a reliability-aware, OpenAI-compatible gateway: canary routing, rate limits, admission control, circuit breaking, retry and fallback.
- Make every layer observable: SLIs (TTFT, TPOT, tokens, cost), model-server and GPU metrics, traces and logs.
- Detect SLO violations, diagnose them with evidence, and remediate safely with a human in the loop.

1.2 Non-goals (current version)

- Multi-region or multi-cluster serving; pretraining; RLHF / preference optimisation.
- Fully autonomous remediation without approval.
- Pod lifecycle management. The deploy controller owns routing; Kubernetes (Helm/KEDA) owns pods.

1.3 Terminology

Term	Meaning
Deployment	A named serving endpoint (e.g. qwen-farm) with a stable version and an optional canary version
Model version	Immutable registry entry: base model + adapter + dataset hash + eval metrics + lifecycle stage
TTFT / TPOT	Time to first token / time per output token: the two latencies users perceive in streaming
Revision	Monotonic counter on a deployment, bumped by every mutation; used for optimistic concurrency
Incident	Human-facing record of one run of the agent's state machine, keyed by incident id
Ship gate	Policy comparing two evaluated versions → SHIP_CANARY / NEEDS_REVIEW / BLOCK

2. High-level design

2.1 System context

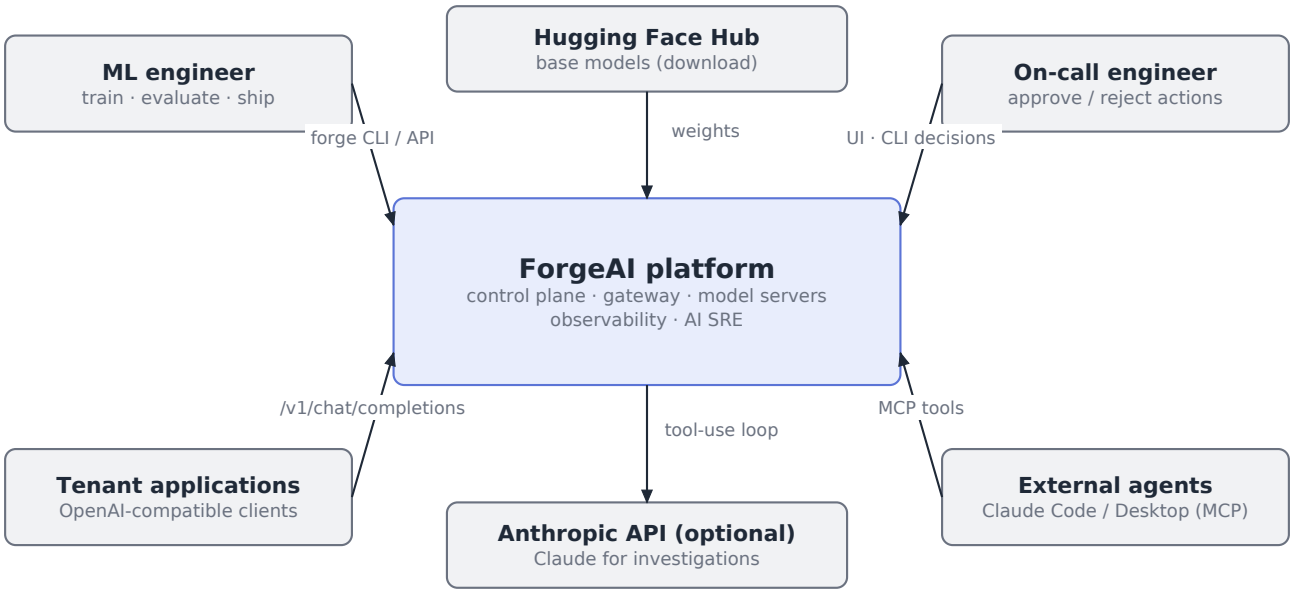


Figure 1. System context. Everything inside the platform box is self-hosted; Claude is optional and the agent falls back to a deterministic runbook without it.

2.2 Architecture

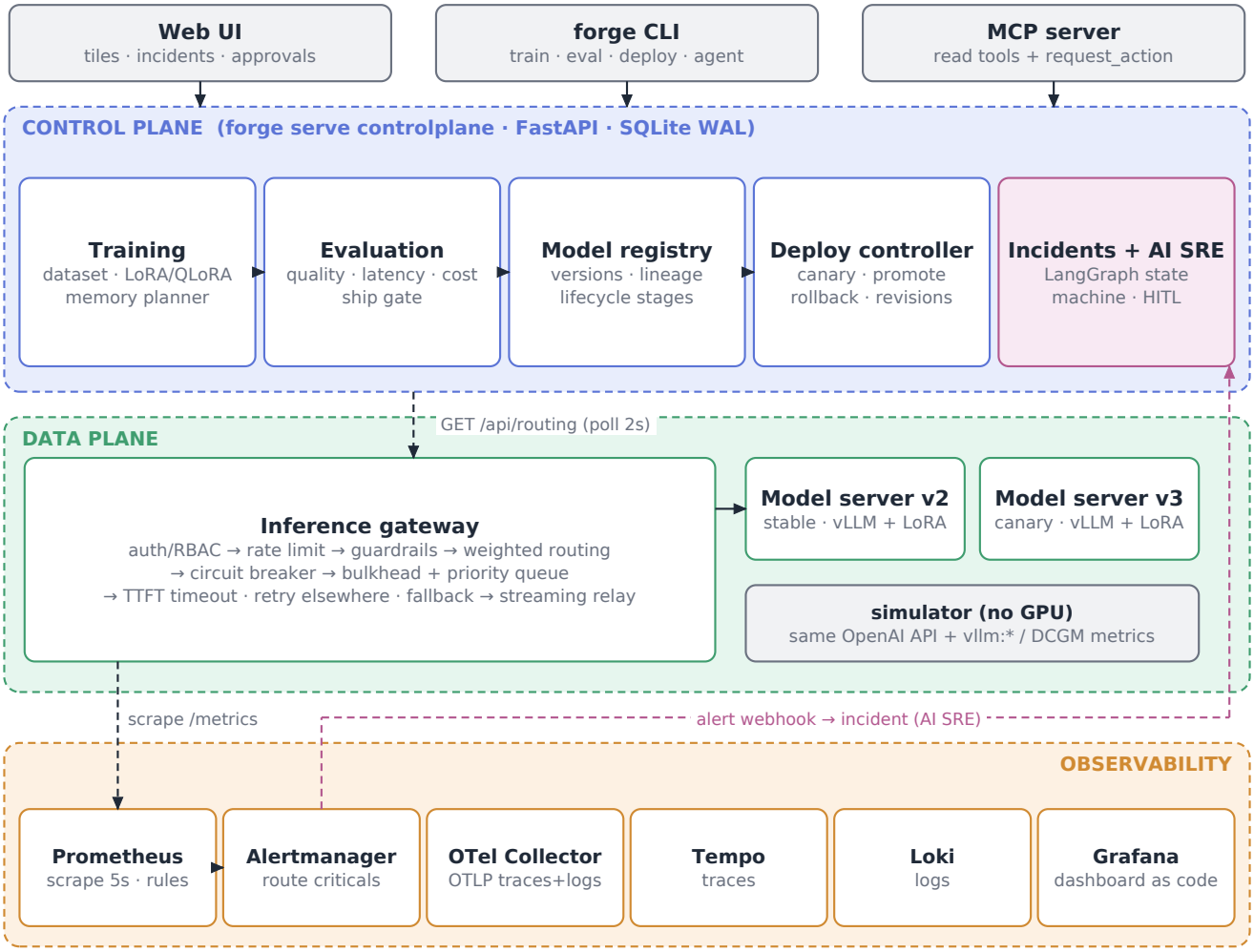


Figure 2. Logical architecture. Solid arrows are request paths; dashed arrows are asynchronous (polling, scraping, alerting).

2.3 Components

Component	Responsibility	Technology	Scaling / state
Training	Dataset validation and lineage; SFT with LoRA/QLoRA; GPU memory plan	transformers, peft, bitsandbytes	Batch job on a GPU node; writes immutable run dirs
Evaluation	Quality, TTFT, p95, tokens/s, $/request on an eval set; ship gate	httpx (streaming)	Stateless; reports to registry
Model registry	Immutable versions, metrics, stages, transition history	SQLite (WAL); MLflow mirror	Single writer
Deploy controller	Desired routing state; canary/promote/rollback; revisions; idempotency; events	SQLite	Single writer
Control-plane API	REST for all of the above; incidents; alert webhook; UI	FastAPI	1 replica (SQLite)
Inference gateway	OpenAI-compatible serving with reliability controls and SLI metrics	FastAPI, httpx	Stateless; HPA on CPU
Model servers	Token generation with continuous batching and LoRA adapters	vLLM (simulator locally)	One Deployment per version; KEDA on queue
AI SRE	Investigate with tools, propose action, wait for approval, execute, verify	LangGraph + SQLite checkpoints; Claude or runbook	Thread pool in control plane
MCP server	Expose read tools and request_action to external agents	mcp 2.x (MCPServer)	stdio / streamable-http
Observability	Metrics, alerts, traces, logs, dashboards	Prometheus, Alertmanager, OTel, Tempo, Loki, Grafana	Standard deployments

2.4 Control plane / data plane split

The control plane owns **desired state** (which versions serve which share of traffic). The gateway converges on it by polling `/api/routing` every 2 s, so rollouts are **eventually consistent** with a bounded lag. If the control plane is unreachable the gateway keeps serving its **last-known-good** routing table and exports `forge_routing_stale_seconds`; an alert fires after 30 s of staleness. The data plane never depends on the control plane being up to serve traffic.

2.5 Deployment views

Concern	Local (docker-compose)	Kubernetes (Helm chart)
Model servers	3 simulator containers (v1, v2, v3) with per-version capacity profiles	vLLM Deployment per version: --enable-lora --lora-modules, GPU limits, nodeSelector, tolerations
Version labels on metrics	Static scrape labels in prometheus.yml	PodMonitor relabels pod labels forgeai.dev/model, forgeai.dev/version
Autoscaling	n/a	Gateway HPA (CPU); KEDA ScaledObject on vllm:num_requests_waiting per version
Availability	restart: unless-stopped	Gateway PDB, topology spread, preStop drain, 60 s grace
Isolation	Compose network	NetworkPolicy: model servers reachable only from gateway, control plane, monitoring
Alerts	deploy/prometheus/alerts.yml	PrometheusRule from the same file (test keeps copies identical)
Secrets	.env (dev keys)	Secret / existingSecret for API keys; optional Anthropic + HF token secrets

2.6 Non-functional requirements

Area	Requirement	How it is met
Latency SLO	TTFT p95 ≤ 500 ms; e2e p95 ≤ 3 s	Recording rules + InferenceTTFTSLOViolation (30 s)
Availability	Error rate ≤ 2%; canary failures invisible to users	Retry to a different backend before first token; breaker; fallback to stable
Overload	Degrade predictably; protect paying tenants	Bulkhead + bounded priority queue; 429/503 with Retry-After
Safety of change	No production change without a human decision	Write tools only via approval gate; RBAC operator role
Auditability	Who changed what, when, why	audit table, deployment_events, stage_transitions, incident timeline
Recoverability	Agent runs survive restarts	LangGraph SQLite checkpointer; resume by incident id
Observability	Every request attributable by version and tenant	Labelled histograms/counters; trace spans; x-trace-id header

2.7 Key design decisions

Decision	Alternative considered	Rationale
Gateway polls control plane for routing	Push / watch stream	Polling is trivially robust to control-plane restarts; last-known-good table gives graceful degradation
Metrics pull (Prometheus); OTLP only for traces + logs	OTLP metrics push	vLLM, DCGM and kube-state-metrics speak Prometheus; one TSDB for every layer
Retry only before first token, to a different backend first	Retry same backend; retry mid-stream	Mid-stream retry duplicates tokens; retrying the failing canary wastes the latency budget
Slow canary handled by agent, not breaker	Trip breaker on latency	Breaker trips on errors / TTFT timeout (outage); a 3× slowdown is a regression needing diagnosis
Agent proposes; graph executes	LLM calls write tools directly	Contains prompt injection: worst case is a wrong recommendation shown to a human
Separate propose and await_approval nodes	One node	LangGraph re-runs an interrupted node on resume; side effects before interrupt() would duplicate
SQLite (WAL) state store	Postgres	Zero-ops for one replica; plain SQL keeps migration a connection-string change
Simulator with a capacity model	Echo/mock server	Incidents emerge from load (queueing, KV pressure) instead of being scripted
Ship gate as policy with budgets	Leaderboard / single metric	Quality-vs-latency-vs-cost is a decision; budgets make it explicit and reviewable

2.8 Security model

- **Authentication:** bearer API keys mapped to (tenant, role). **Roles:** viewer < operator < admin. Inference and reads need viewer; any mutation, approval or webhook needs operator.
- **Guardrails** before any model sees input: message shape, prompt size cap, credential detection (AWS keys, private keys, API keys), max_tokens bounds.
- **Agent containment:** read-only investigation tools; tool output wrapped as untrusted data; the only LLM output channel is a strict `submit_diagnosis` schema whose action is an enum; the graph re-validates the action, strips unknown args and refuses cross-deployment targets.
- **Idempotent writes:** agent actions carry `incident_id:action` keys; API callers may send `Idempotency-Key`.
- **Network:** model servers are reachable only from gateway, control plane and monitoring (NetworkPolicy); containers run as non-root with read-only root filesystems.

3. Low-level design

3.1 Code layout

Module	Lines	Key types / functions
forge/config.py	96	Settings (env-driven), parsers for API keys and tenant limits
forge/db.py	114	SCHEMA, connect() (WAL), tx() (BEGIN IMMEDIATE), audit()
forge/telemetry.py	66	setup(service, app): tracer provider, OTLP spans + logs, instrumentation
forge/training/dataset.py · train.py	69 · 232	prepare(); TrainConfig, memory_plan(), train()
forge/evaluation/harness.py · scorers.py · compare.py	171 · 39 · 83	run_eval() → EvalReport.headline(); exact_match/contains/token_f1; ShipPolicy, compare()
forge/registry/registry.py	160	Registry.register/transition/update_metrics/history; ALLOWED
forge/deploy/controller.py	215	DeployController.create/start_canary/set_weight/promote /rollback/pause/scale/routing; _mutate()
forge/gateway/*.py	~580	app (chat_completions, Gateway.open_upstream), routing (RoutingTable.plan), admission (Bulkhead), breaker, ratelimit, retry, guardrails, auth, metrics
forge/serving/simulator.py	287	Profile, Faults, Engine (acquire/release, kv_usage), completions()
forge/agent/tools.py · prom.py · cost.py	257 · 94 · 79	Toolbox (17 tools), Prom/Tempo/Loki, explain_cost_change()
forge/agent/investigator.py · graph.py · incidents.py	326 · 178 · 64	Heuristic/ClaudeInvestigator, Diagnosis; IncidentGraph; IncidentStore
forge/controlplane/app.py · static/index.html	317	REST API, webhook, overview, UI
forge/mcp_server.py · cli.py	85 · 430	MCPServer registration; Typer app

3.2 Data model (SQLite)

Table	Columns	Notes
model_versions	`name, version (PK), base_model, adapter_uri, serving_url, dataset_hash, run_id, stage, metrics JSON, tags JSON, created_at`	Immutable except stage, metrics, serving_url
stage_transitions	`id, name, version, from_stage, to_stage, actor, reason, at`	Append-only history
deployments	`name (PK), model, stable_version, canary_version, canary_weight, status, revision, replicas, updated_at`	status ∈ stable | canary | paused
deployment_events	`id, deployment, action, detail JSON, actor, revision, at`	Answers “what changed?” for the agent
incidents	`id (PK), title, severity, state, source, labels JSON, diagnosis JSON, proposed_action JSON, timeline JSON, created_at, updated_at`	Projection of the graph run; labels.fingerprint de-duplicates alerts
idempotency	`key (PK), response JSON, at`	Stored in the same transaction as the mutation
audit	`id, actor, action, target, detail JSON, at`	Every API mutation and approval

LangGraph checkpoints live in a separate SQLite file (agent_checkpoints.db), keyed by thread_id = incident id.

3.3 Model registry lifecycle

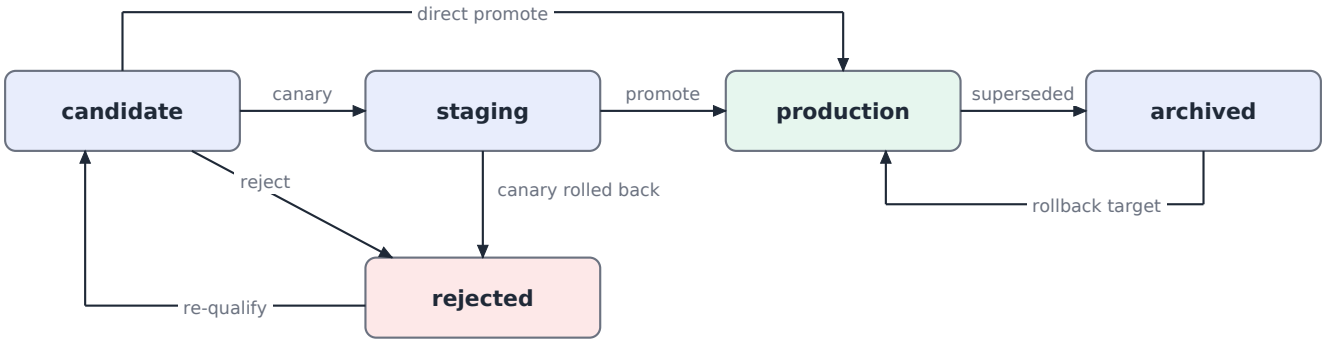


Figure 3. Stage transitions (Registry.ALLOWED). Exactly one production version per model: promoting archives the previous one in the same transaction. Rejected/archived versions cannot be canaried until re-qualified.

3.4 Deploy controller

Every mutation goes through `_mutate(name, op, actor, expected_revision, idempotency_key)`:

```
1. if idempotency_key exists in idempotency table -> return stored response (no-op)
2. BEGIN IMMEDIATE                                  (take the write lock up front)
3. load deployment; if expected_revision != revision -> DeployError("revision conflict")
4. action, detail = op(conn, deployment)             (validates + UPDATEs)
5. revision += 1; updated_at = now
6. INSERT deployment_events(action, detail, actor, revision)
7. INSERT idempotency(key, response)                 (same transaction as the change)
8. COMMIT; then registry side effects (staging / production / rejected transitions)
```

Operation	Preconditions	Effect
create	version has serving_url; name unused	stable = version; registry → production
start_canary	not stable; not rejected/archived; no other canary; weight ∈ (0,100]	canary = version, weight, status canary; candidate → staging
set_weight	canary exists; weight ∈ [0,100]	canary_weight = weight
pause	status canary	status paused (weights frozen)
promote	canary exists	stable = canary; canary cleared; registry → production (old archived)
rollback	canary exists, or a previous stable is known	abort canary (→ rejected) or revert stable to the version it replaced (from promote events)
scale	replicas ∈ [1,64]	replicas recorded (Kubernetes/KEDA act on it)

routing() produces, per deployment: `{revision, status, backends: [{model, version, url, weight, role, price_per_1k_tokens}], fallback: stable}`. Stable weight is 100 − canary weight; a canary at weight 0 is omitted.

3.5 Gateway request pipeline

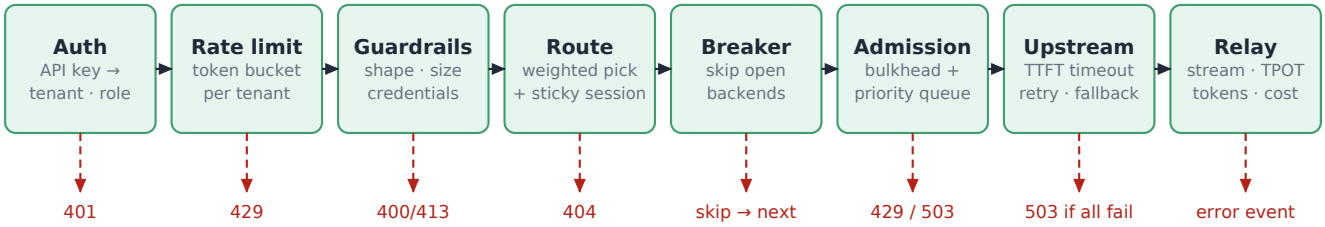


Figure 4. Gateway stages in order; red dashed arrows are the rejection or failure outcome of each stage.

3.6 Gateway algorithms

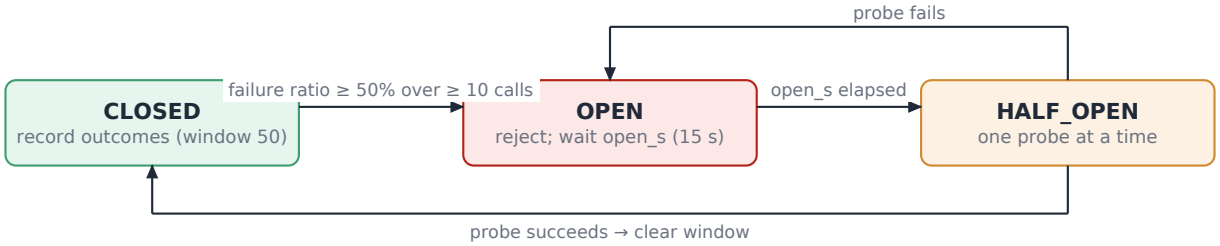
Token bucket (per tenant)

tokens = min(burst, tokens + elapsed × rate); admit if tokens ≥ 1, else 429 with Retry-After = (1 − tokens) / rate. Limits come from `FORGE_TENANT_LIMITS` as rate/burst/priority, defaulting to platform 100/200/2, acme 20/40/1, free-tier 5/10/0.

Admission control (bulkhead per backend)

Each backend (model:version) has a semaphore of max_inflight (32) and a bounded queue (64). A priority class may only occupy a share of the queue: **priority 0 → 50%**, **1 → 80%**, **2 → 100%**. When in-flight is full and the queue is beyond the caller's share, the request is shed with 429 (queue_full). A request that waits longer than queue_timeout (5 s) gets 503 (queue_timeout). Under pressure the free tier is shed first.

Circuit breaker (per backend)



Failures counted: connect errors, HTTP 5xx / 429 from the model server, TTFT timeout (10 s), and streams that break mid-response.

Routing, retry and fallback

`plan(deployment, sticky_key, attempts)` picks a primary by weighted random (seeded by `x-session-id` for stickiness) and returns **[primary, other backends…, fallback, primary]** truncated to retries + 1 (3). Attempts skip backends whose breaker is open, sleep full-jitter backoff `uniform(0, min(2 s, 0.1 × 2^n))` between attempts, and are allowed only until the first content token has been received. After that, the response is committed: a broken stream emits an in-band error chunk and counts as a breaker failure. A request served by a non-primary backend increments `forge_fallback_total{from_version, reason}`.

Streaming relay accounting

The gateway always requests streaming upstream (with `include_usage`), even for non-streaming clients, so TTFT is measured the same way for everyone. On completion it records: e2e duration, TPOT = (t_end − t_first) / (completion_tokens − 1), prompt and completion tokens per tenant, cost = tokens / 1000 × price_per_1k_tokens of the version that served, and the status code. Response headers: `x-forge-deployment, x-forge-version, x-forge-route (stable|canary|fallback), x-trace-id`.

3.7 Model-server simulator capacity model

```
admission     : at most max_num_seqs * capacity_multiplier running; others wait  ->
vllm:num_requests_waiting
kv_usage      : resident_tokens / kv_capacity_tokens                          ->
vllm:gpu_cache_usage_perc
prefill (TTFT): (ttft_base + prefill_ms_per_1k * prompt_tokens/1000) * ttft_mult * (1 + 1.5 *
kv_usage^2)
decode (TPOT) : tpot_ms * tpot_mult * (1 + batch_penalty * (running-1)) * (1 + kv_usage)
quality       : question rank (version-independent hash) < round(accuracy * N)  -> correct answer
GPU           : util ~ 90 * running/capacity ; FB used = mem_base + extra_mem + 6 GB * kv_usage
```

v3 differs from v2 only in its profile (max_num_seqs 4 vs 16, KV capacity 1.8k tokens, +4.5 GB base memory). At evaluation concurrency 4 it never queues, so it passes the ship gate. At 50% of production traffic it queues, which is the regression the agent must find.

3.8 Evaluation and ship gate

`run_eval()` streams every eval prompt (bounded concurrency), measuring TTFT, latency and usage. Headline metrics written to the registry: accuracy (primary scorer, default contains), token_f1, ttft_p95_s, latency_p50/p95_s, tokens_per_s (decode), avg_output_tokens, cost_per_1k_tokens, cost_per_request, error_rate. Cost model: `$/1k tokens = (gpu_hourly_usd / 3600) / aggregate_tokens_per_s × 1000`. A GPU costs the same per hour whether busy or idle, so cost per token is set by throughput.

Rule (ShipPolicy defaults)	Outcome
Accuracy drops more than 1 pt, or error rate > 2%	BLOCK
p95 latency +>20%, TTFT p95 +>25%, or cost/request +>25%, and accuracy gain ≥ 2 pts	NEEDS_REVIEW (trade-off stated in the reason)
Same regressions with accuracy gain < 2 pts	BLOCK
Otherwise	SHIP_CANARY

3.9 Training

- **Data:** accepts `messages` or `prompt/response` rows; validates roles and a final assistant turn; removes duplicates; deterministic seeded split; SHA-256 of the raw file recorded as lineage.
- **Loss masking:** each turn's text is the diff between the chat-template render before and after it. Role headers and system/user turns get label −100; assistant content plus its end-of-turn token are trained. Non-prefix-stable templates are rejected.
- **Loop:** AdamW on LoRA params, cosine schedule with warmup, gradient accumulation, grad-clip 1.0, gradient checkpointing, periodic val loss and checkpoints; QLoRA loads NF4 double-quantised weights.
- **Memory plan:** weights (2 B/param bf16, ~0.55 NF4) + LoRA params × 16 B (weights, grads, Adam m/v, fp32 master) + activations (batch × seq × hidden × layers × 2 with checkpointing, 34 without) + logits (batch × seq × vocab × 4 B) + 1 GB overhead. LoRA params use real projection shapes (GQA k/v, MLP intermediate size).

3.10 Agent

Tool	Risk	Source
`get_slo_status, get_service_health, get_model_metrics`	read	Prometheus (gateway SLIs by deployment / version)
`get_gpu_metrics, get_queue_status, get_recent_errors`	read	Prometheus (vLLM, DCGM, gateway), Loki
`get_deployment_status, compare_model_versions`	read	Controller state + events; registry eval metrics
`search_slow_traces, get_trace`	read	Tempo
`explain_cost_change`	read	Prometheus increase() over window vs offset window
`rollback_model, pause_rollout, set_canary_weight, start_canary, promote_canary, scale_inference`	write	DeployController via approval gate only

Investigators

- **ClaudeInvestigator** (claude-opus-5-5, adaptive thinking, effort high, server-side refusal fallback): a manual tool-use loop over read tools only, at most 12 turns, ending in a strict `submit_diagnosis` call (action ∈ rollback_model, pause_rollout, set_canary_weight, scale_inference, none).
- **HeuristicInvestigator** (runbook): SLO → what changed → suspect vs baseline (TTFT ratio > 1.5 or error rate > max(2× baseline, SLO)) → rollback; else saturation (queue or GPU > 85%) → scale +1; else escalate. Used when Claude is unavailable, and as the fallback when the LLM path errors, refuses or never submits.

Incident state machine

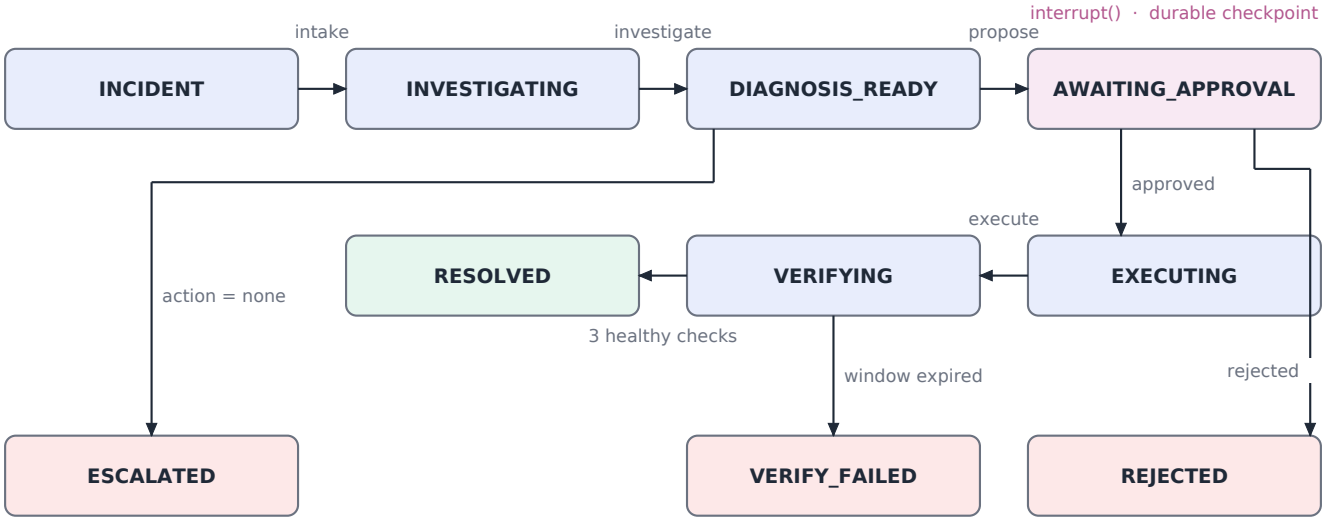


Figure 5. Graph nodes: intake → investigate → propose → await_approval (interrupt) → execute → verify; escalate when no safe action exists. Every transition is mirrored to the incidents table.

IncidentState field	Content
`incident_id, deployment, alert`	Thread key; target deployment; alert labels/annotations (untrusted)
`phase`	Current state name
`diagnosis`	summary, root_cause, evidence[], confidence, suspected_change, action, action_args, rationale, investigator, tool_calls[]
`decision`	{approved, by, comment} supplied on resume
`action_result, verification`	Controller response; last SLO snapshot and ok flag
`log`	Append-only (operator.add reducer) node log

Execute-time checks

- Action must be in the allowlist and be a write tool; none never reaches execute.
- `action_args.deployment` must equal the incident's deployment; unknown args are dropped.
- Idempotency key `{incident_id}:{action}`; actor recorded as `agent/approved-by:{who}`.
- Verify: poll SLOs every 5 s within FORGE_VERIFY_WINDOW_S (150 s in compose); 3 consecutive in-SLO checks → RESOLVED.

3.11 APIs

Method & path	Role	Purpose
`GET /api/models · POST /api/models`	viewer · operator	List / register versions
`POST /api/models/{n}/{v}/metrics | /stage | /serving-url`	operator	Record eval metrics; transition; set endpoint
`GET /api/models/{n}/history · /compare?base&candidate;`	viewer	Stage history; ship-gate comparison
`GET /api/routing`	viewer	Gateway routing table
`GET|POST /api/deployments · GET /{name}/events`	viewer · operator	List / create; event log
`POST /api/deployments/{name}/{canary|weight|promote|rollback|pause|scale}`	operator	Rollout operations; Idempotency-Key header; expected_revision in body
`GET /api/incidents[/{id}] · POST /api/incidents`	viewer · operator	List / detail with graph state; open (optional proposal)
`POST /api/incidents/{id}/decision`	operator	Approve / reject; resumes the graph
`POST /webhooks/alertmanager`	operator	Firing alerts → de-duplicated incidents
`POST /api/agent/ask · GET /api/overview`	viewer	Free-form question; UI data
`Gateway: POST /v1/chat/completions · GET /v1/models`	viewer	OpenAI-compatible inference
`Model server: /v1/chat/completions, /v1/models, /health, /metrics, /admin/faults`	(internal)	vLLM-compatible; faults only on the simulator
`/healthz, /metrics on every service`	none	Probes and scraping

3.12 Metrics and alerts

Metric	Type · labels
`forge_requests_total`	counter · deployment, version, tenant, code
`forge_request_duration_seconds, forge_ttft_seconds, forge_tpot_seconds`	histogram · deployment, version
`forge_tokens_total, forge_cost_usd_total`	counter · deployment, version, tenant (+ kind)
`forge_inflight_requests, forge_queue_depth, forge_breaker_state`	gauge · backend
`forge_rejected_total, forge_fallback_total, forge_retries_total, forge_upstream_errors_total`	counter · reason / from_version / backend, kind
`forge_routing_revision, forge_canary_weight, forge_routing_stale_seconds`	gauge
`vllm:num_requests_running|waiting, vllm:gpu_cache_usage_perc, vllm:time_to_first_token_seconds, …`	model server (vLLM names)
`DCGM_FI_DEV_GPU_UTIL, DCGM_FI_DEV_FB_USED, DCGM_FI_DEV_FB_FREE`	GPU (DCGM exporter names)

Alert	Condition	Severity → route
`InferenceTTFTSLOViolation`	forge:ttft_p95:1m > 0.5 and traffic > 0.2 rps, for 30 s	critical → agent
`InferenceErrorRateHigh`	forge:error_ratio:1m > 2%, for 1 m	critical → agent
`InferenceQueueBacklog`	queue depth > 10, for 1 m	warning
`CircuitBreakerOpen`	breaker state = 2, for 15 s	warning
`GatewayRoutingStale`	routing stale > 30 s, for 1 m	warning

Alert	Condition	Severity → route
`GPUMemoryPressure`	FB used / total > 90%, for 2 m	warning

4. Flows

Sequence diagrams for the main paths. Solid arrows are calls, dashed arrows are responses, red arrows are failures, yellow boxes are notes.

F1. Model lifecycle: dataset to production

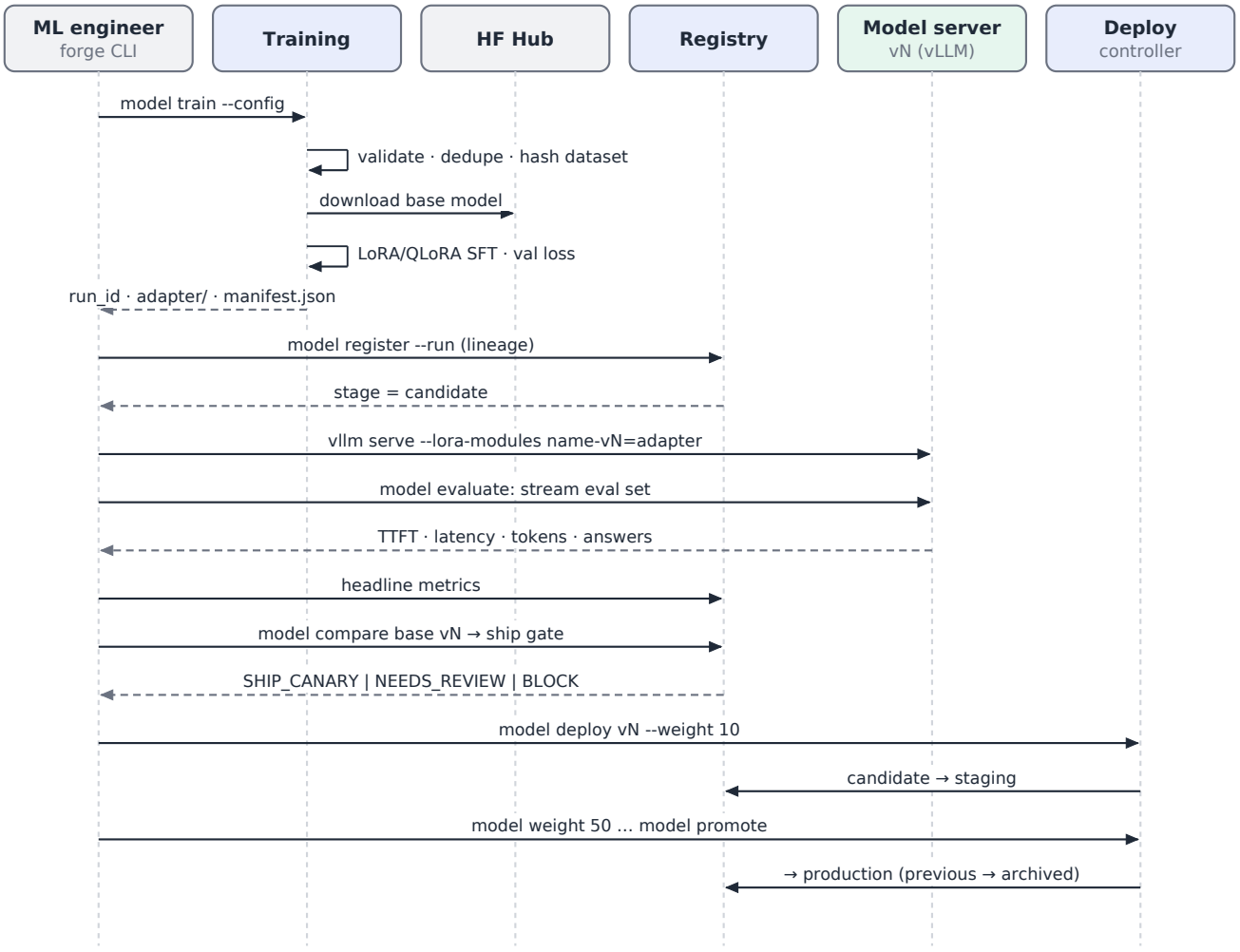


Figure 6. Train → register → evaluate → gate → canary → promote. BLOCK stops the flow; NEEDS_REVIEW requires an explicit human trade-off decision.

F2. Inference request (happy path, streaming)

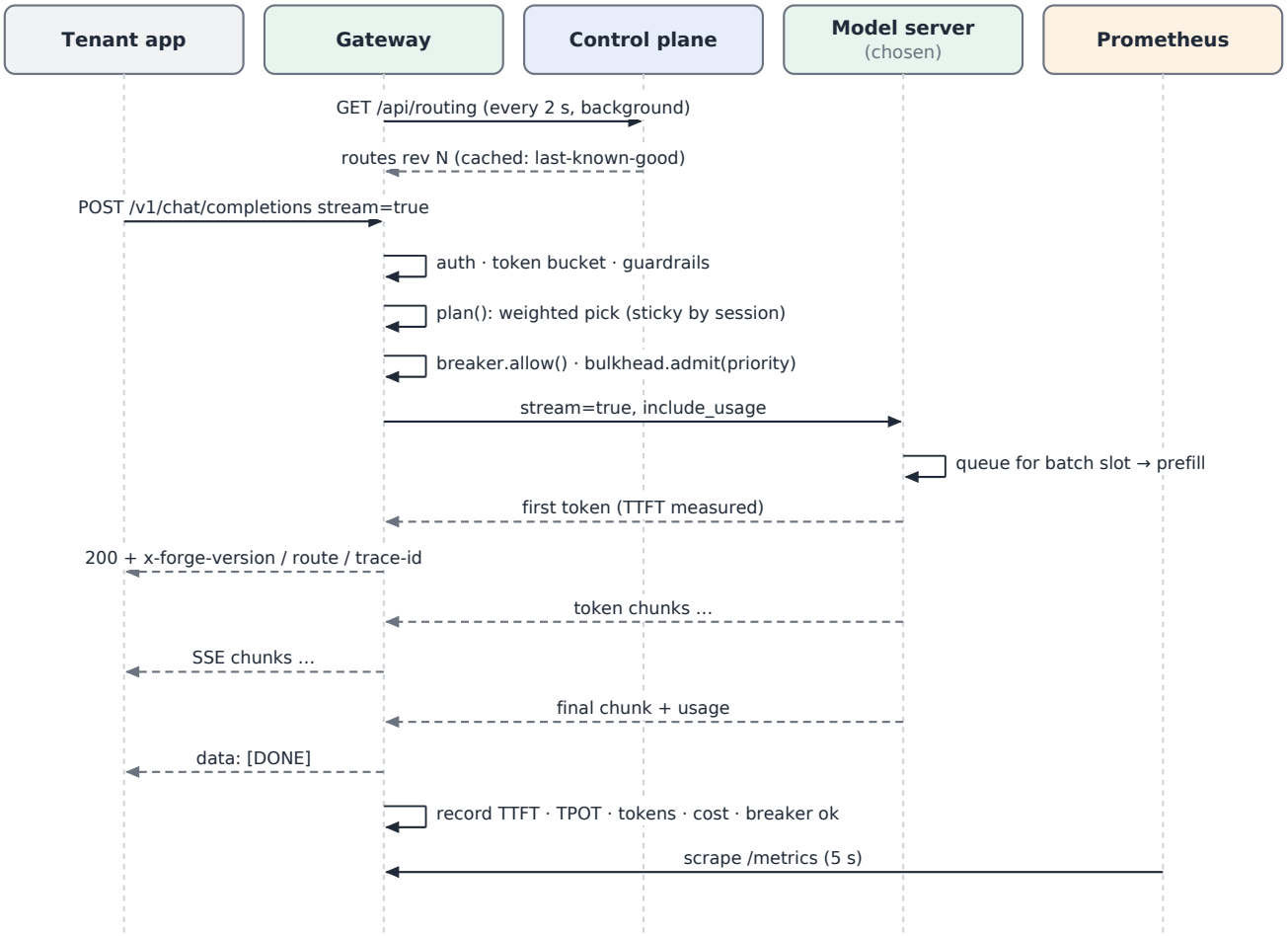


Figure 7. The control plane is not on the request path; the gateway serves from its cached table.

F3. Canary failure: retry elsewhere, fallback, breaker

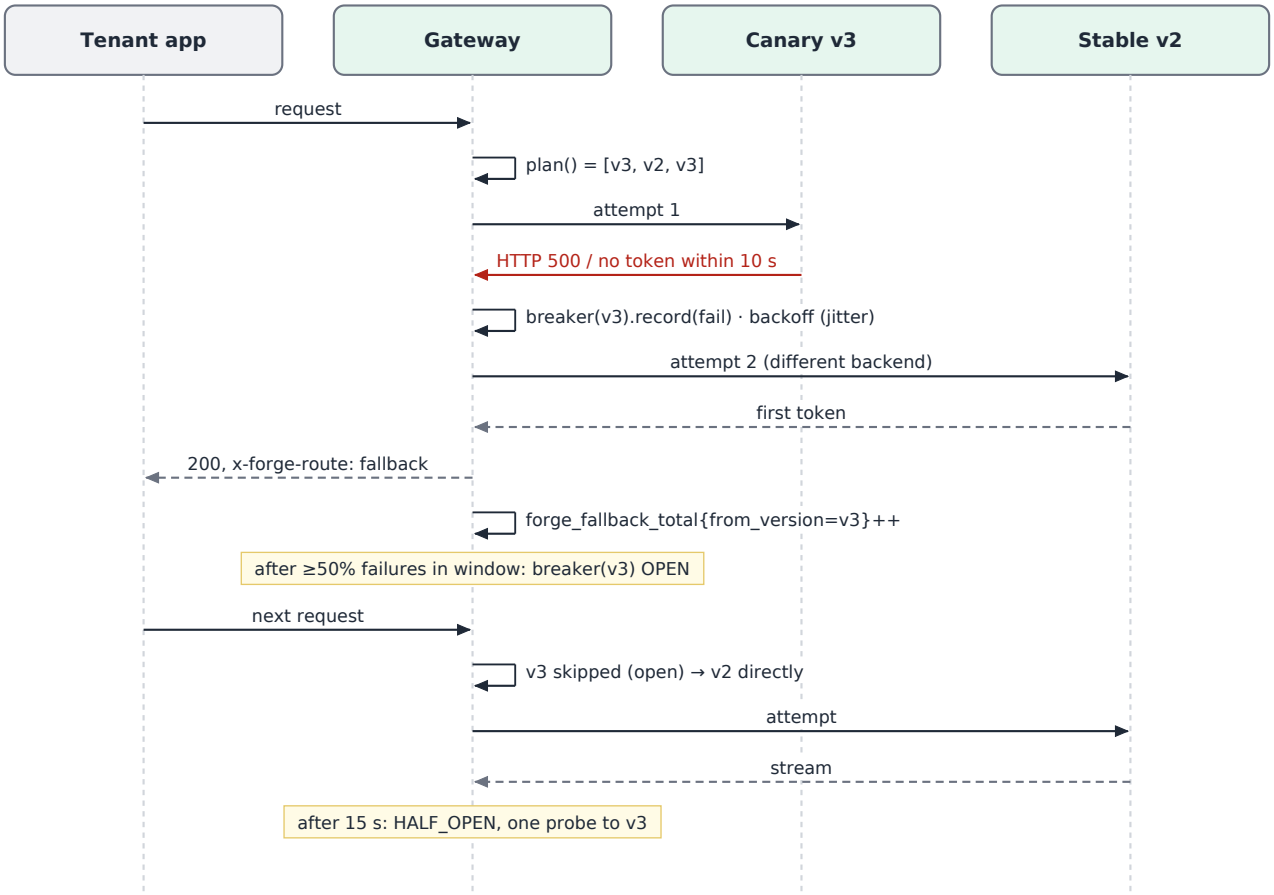


Figure 8. Users never see the canary's failures: verified by the integration test (200 requests, zero non-200 responses while v3 returns 500).

F4. Incident: detection to verified recovery

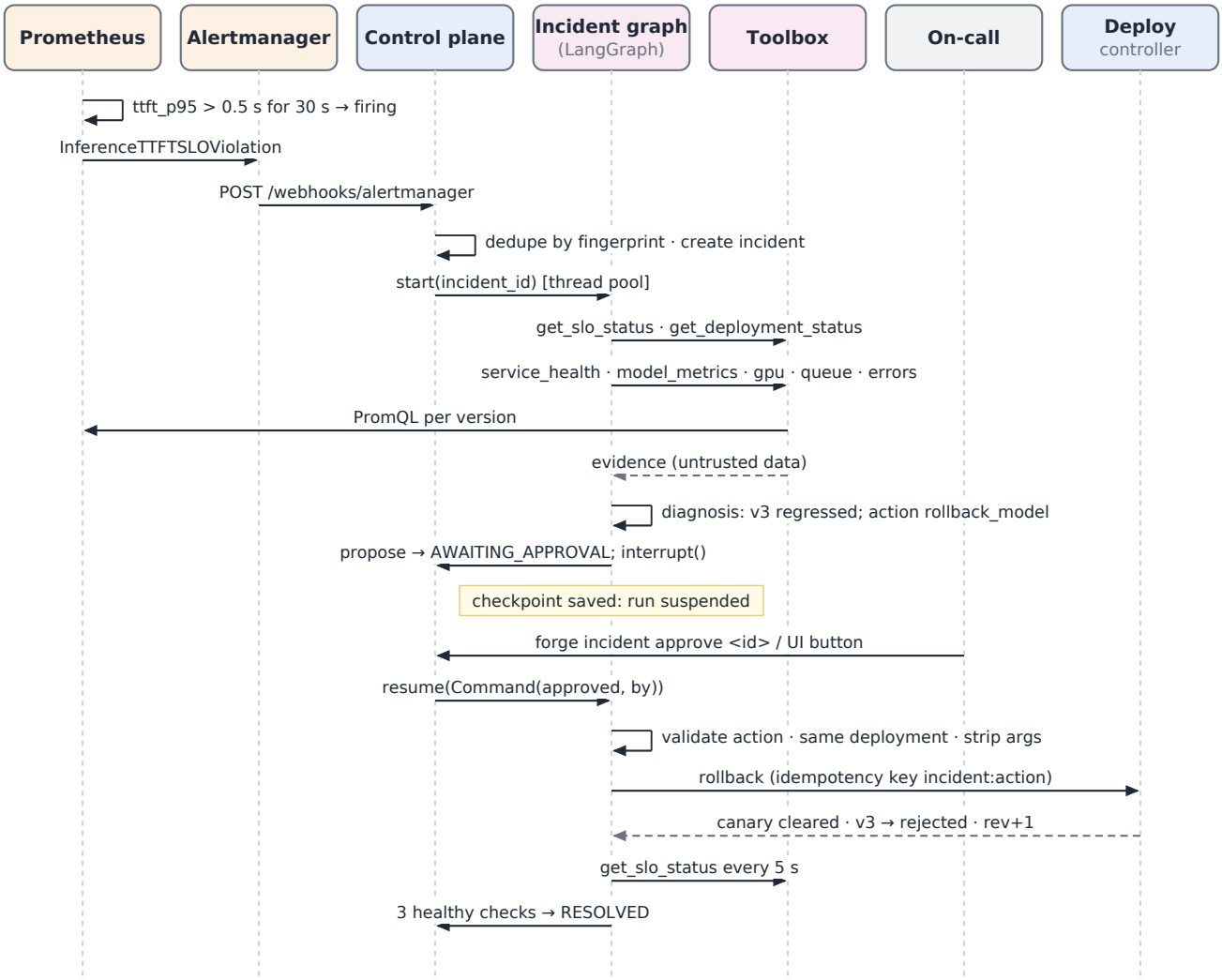


Figure 9. Measured on the local stack: alert to diagnosis in seconds, approval to verified recovery ~2 minutes (bounded by the 1-minute rate window draining).

F5. Cost attribution: “why did inference cost go up?”

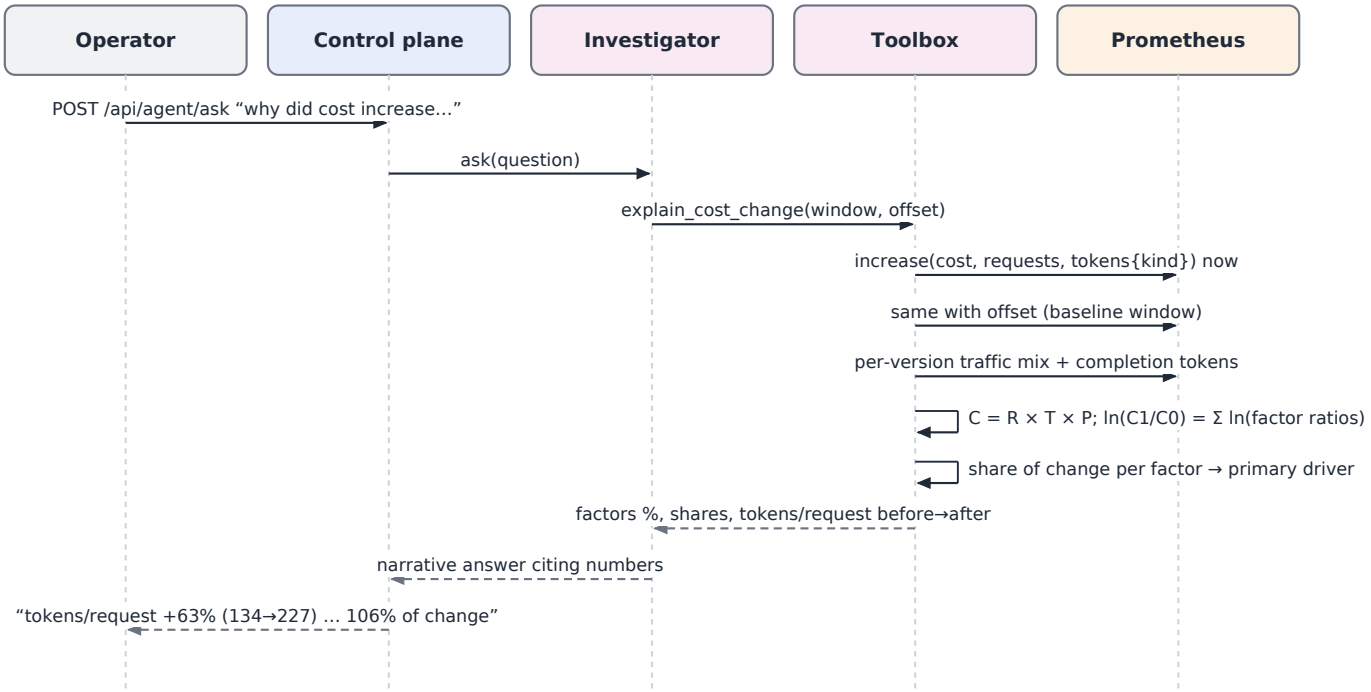


Figure 10. Log decomposition makes factor contributions exactly additive; plain percentage deltas do not add up once changes are large.

F6. External agent via MCP: same approval gate

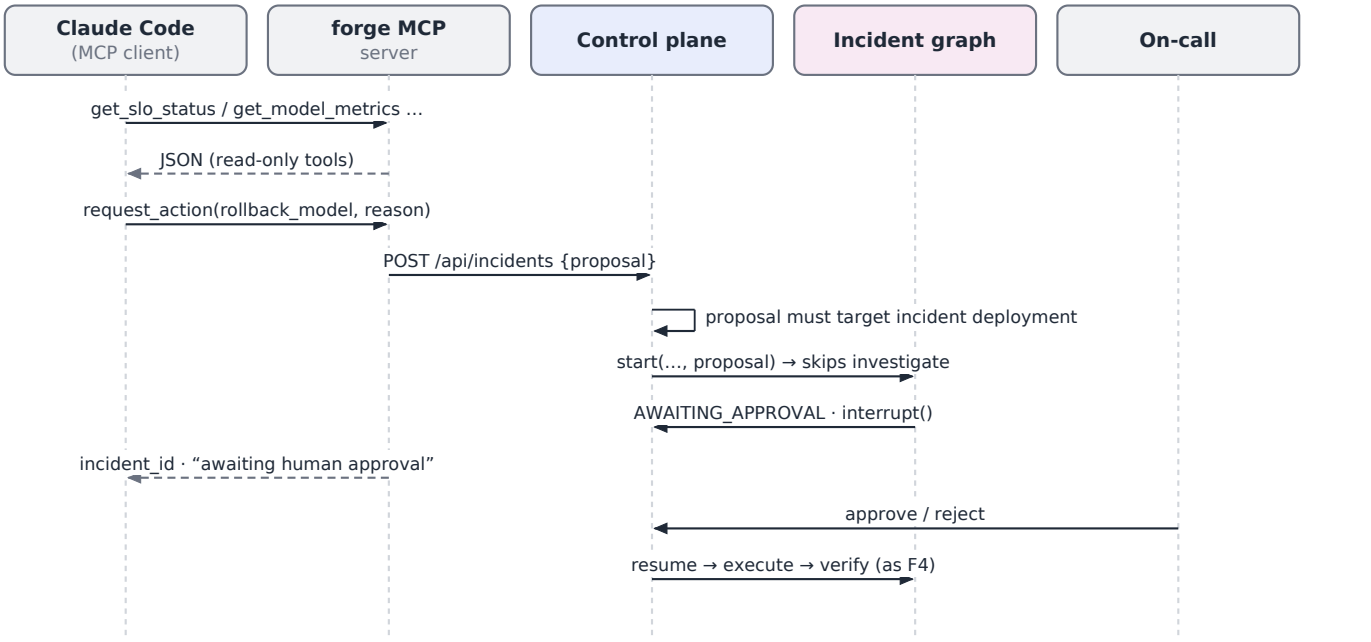


Figure 11. No MCP tool can mutate production directly; one HITL policy covers every agent.

F7. Control-plane outage: graceful degradation

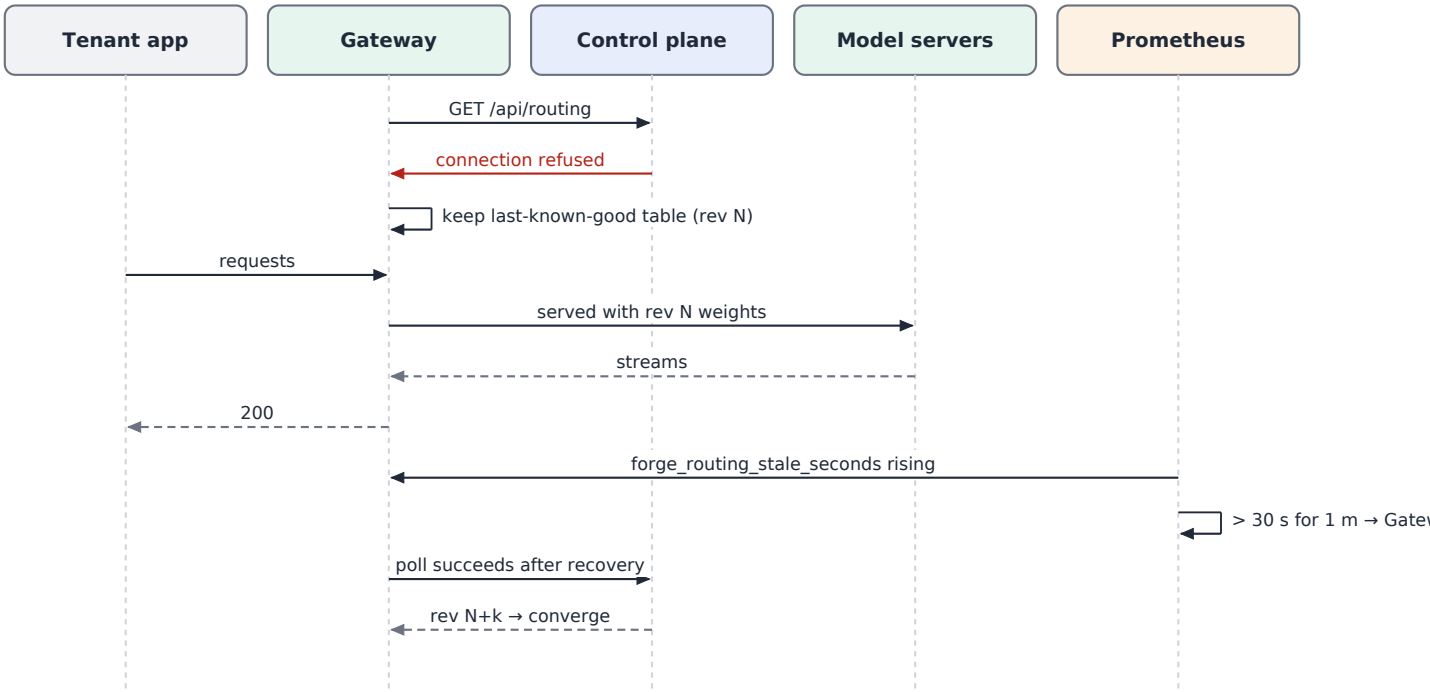


Figure 12. Rollout changes pause during the outage; traffic does not. On a cold start with no table, the gateway can load FORGE_ROUTING_FILE.

F8. Overload: priority-aware load shedding

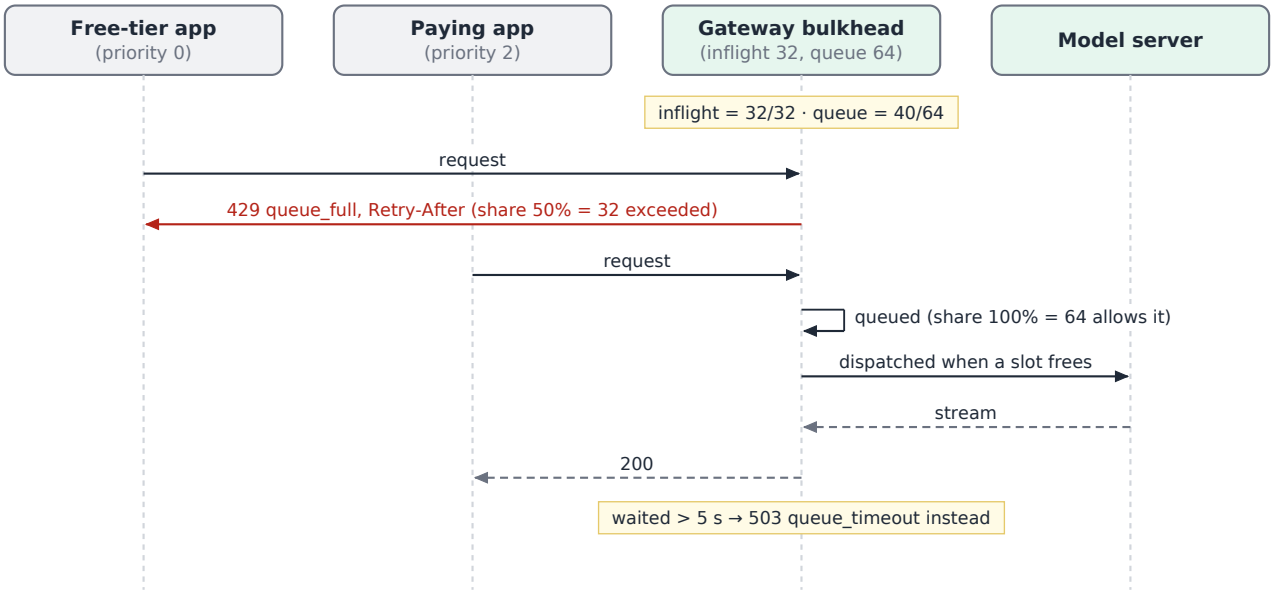


Figure 13. Backpressure is explicit (429/503 with Retry-After) and ordered by tenant priority.

5. Verification status and known limits

5.1 Verified

What	How	Result
Registry, controller, gateway primitives, ship gate, cost attribution, dataset prep	Unit tests	Pass
Canary split, streaming, auth, guardrails, fallback, breaker, metrics	Integration test: real simulator + gateway processes	Pass; 0 user-visible errors with canary at 100% 5xx
Incident lifecycle incl. resume from checkpoint in a new process, rejection, cross-deployment refusal	Agent tests (fake Prometheus)	Pass
Alert → diagnosis → approval → rollback → verify	Docker stack, real Prometheus/Alertmanager, run twice	RESOLVED both times
Cost attribution on live data	Output tokens ×1.9 on production for 5 min	Driver correctly named: tokens/request
Training loop, loss masking, memory planner	CPU run on SmolLM2-135M; label decode; PEFT param count	Adapter saved; masking exact; 4.88M = 4.88M
Helm chart	helm lint + template (16 resources)	Pass

5.2 Not yet verified

- Claude investigator against the live API (no key in the build environment); the runbook path was used.
- Real vLLM on GPU; QLoRA (requires CUDA + bitsandbytes); MLflow mirroring; Helm install on a cluster.

5.3 Known limits and next steps

Limit	Impact	Next step
SQLite state store	Single control-plane replica	Postgres; same SQL
In-process rate limits	Limits are per gateway replica	Redis-backed buckets behind the same interface
Controller manages routing, not pods	Model servers deployed separately	Operator or GitOps (Argo CD) reconciling modelServers from the registry
Reference-match eval scoring	Weak signal for open-ended answers	LLM-as-judge / pairwise scorers in SCORERS
Agent has no cross-incident memory	Repeats investigation work	Store past diagnoses; retrieve by deployment + symptom
Single-region	No regional failover	Per-region gateways with global routing